

Article

YoloMal-XAI: Interpretable Android Malware Classification Using RGB Images and YOLO11

Chaymae El Youssefi *  and Khalid Chougali 

Engineering Sciences Laboratory, Ibn Tofail University, Kenitra 14000, Morocco; khalid.chougali@uit.ac.ma

* Correspondence: chaymae.elyoussefi@uit.ac.ma

Abstract

As Android malware grows increasingly sophisticated, traditional detection methods struggle to keep pace, creating an urgent need for robust, interpretable, and real-time solutions to safeguard mobile ecosystems. This study introduces YoloMal-XAI, a novel deep learning framework that transforms Android application files into RGB image representations by mapping DEX (Dalvik Executable), Manifest.xml, and Resources.arsc files to distinct color channels. Evaluated on the CICMalDroid2020 dataset using YOLO11 pretrained classification models, YoloMal-XAI achieves 99.87% accuracy in binary classification and 99.56% in multi-class classification (Adware, Banking, Riskware, SMS, and Benign). Compared to ResNet-50, GoogLeNet, and MobileNetV2, YOLO11 offers competitive accuracy with at least 7× faster training over 100 epochs. Against YOLOv8, YOLO11 achieves comparable or superior accuracy while reducing training time by up to 3.5×. Cross-corpus validation using Drebin and CICAndMal2017 further confirms the model's generalization capability on previously unseen malware. An ablation study highlights the value of integrating DEX, Manifest, and Resources components, with the full RGB configuration consistently delivering the best performance. Explainable AI (XAI) techniques—Grad-CAM, Grad-CAM++, Eigen-CAM, and HiRes-CAM—are employed to interpret model decisions, revealing the DEX segment as the most influential component. These results establish YoloMal-XAI as a scalable, efficient, and interpretable framework for Android malware detection, with strong potential for future deployment on resource-constrained mobile devices.



Academic Editor: Giorgio Giacinto

Received: 4 June 2025

Revised: 16 July 2025

Accepted: 24 July 2025

Published: 1 August 2025

Citation: El Youssefi, C.; Chougali, K. YoloMal-XAI: Interpretable Android Malware Classification Using RGB Images and YOLO11. *J. Cybersecur. Priv.* **2025**, *5*, 52. <https://doi.org/10.3390/jcp5030052>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: Android malware detection; deep learning; YOLO11; explainable AI (XAI); RGB image representation

1. Introduction

Android remains the most widely used mobile operating system, holding approximately 72.15% of the global market share as of January 2025, with over 3.3 billion active users worldwide [1]. Its dominance, particularly in regions like Africa and South America (over 85% share), has made Android a prime target for cybercriminals who exploit system vulnerabilities to steal data, evade detection, and compromise user privacy.

Recent cybersecurity reports reveal that Trojans constitute over 93% of Android malware, often masquerading as legitimate applications to steal data, hijack devices, and commit financial fraud via phishing emails, fake app stores, and malicious downloads [2]. In 2024, banking Trojans affected over 300,000 users, while ransomware like Vultur—disguised as a security tool—locked nearly 900,000 devices by recording keystrokes and capturing sensitive activity [3].

High-profile incidents underscore the sophistication of these threats. A fake Telegram Premium app harvested user data through phishing sites imitating official platforms [4]. The *Spyrtacus* spyware, embedded in WhatsApp clones, intercepted messages and remotely activated microphones and cameras [5]. In parallel, over 22,800 phishing apps mimicking TikTok, WhatsApp, and Spotify were detected in 2024 alone, targeting user login credentials [6].

To combat Android malware, researchers have developed various detection techniques, broadly categorized into static analysis and dynamic analysis [7]. Static analysis examines an application's source code, permissions, and structural attributes without execution. While computationally efficient, this approach is vulnerable to obfuscation techniques—such as code encryption, packing, and reflection—which allow malware to evade detection by modifying its signature. Dynamic analysis, on the other hand, monitors an application's runtime behavior in a controlled sandbox environment, offering better resilience against obfuscation. Tools such as DroidBox [8] exemplify early efforts in this space by tracking information leaks, file access, and network activity during execution. However, their high computational cost makes them impractical for real-time, large-scale deployment. Additionally, signature-based detection [9], which relies on predefined malware patterns, struggles to identify zero-day attacks and polymorphic malware, where attackers frequently modify the code to bypass detection.

The emergence of machine learning (ML) and deep learning (DL) has significantly improved malware detection capabilities by leveraging large datasets to identify malicious patterns. However, these models face several critical challenges. Obfuscation techniques, such as code packing, encryption, and runtime modifications, enable malware to disguise itself, reducing the effectiveness of ML/DL models [10]. Additionally, deep learning models often function as black-box systems, providing little interpretability. This lack of transparency poses a challenge in cybersecurity applications, where explainability is crucial for trust, regulatory compliance, and decision validation. Furthermore, evasion techniques, such as adversarial attacks, manipulate input features to mislead ML/DL models, significantly reducing their detection accuracy.

To overcome these limitations, image-based malware detection has emerged as a promising direction. Pioneered by Nataraj et al. [11], this method converts binaries into grayscale images for visual pattern recognition. While effective, grayscale images often miss semantic features critical for classification.

Research Gap. Existing image-based malware classifiers, particularly those using grayscale representations or conventional CNNs, lack semantic richness and interpretability—two crucial factors for detecting modern, obfuscated Android malware in real-time scenarios.

To address this gap, this paper introduces YoloMal-XAI, a deep learning framework that integrates YOLO11 and Explainable AI (XAI) for malware classification. Designed for accuracy, efficiency, and real-time deployment, YoloMal-XAI is well-suited for large-scale mobile security applications. The framework leverages RGB image representations, where each color channel encapsulates a distinct malware feature, significantly enhancing detection performance. Furthermore, YoloMal-XAI incorporates Explainable AI (XAI) techniques, such as Grad-CAM, Grad-CAM++, Eigen-CAM, and HiRes-CAM, to provide interpretable insights into the model's decision-making process.

The remainder of this paper is structured as follows:

- Section 2 reviews the literature on image-based Android malware detection and interpretable deep learning.
- Section 3 describes the YoloMal-XAI framework, detailing the dataset, RGB image construction from APK components, and the adaptation of YOLO11 for malware classification.

- Section 4 presents the experimental results, including performance evaluation across binary and multi-class tasks, ablation studies, comparisons with CNNs and YOLOv8, and cross-corpus validation.
- Section 5 discusses the practical implications, generalization, interpretability analysis using multiple XAI techniques, and limitations of the proposed approach.
- Section 6 concludes the study and outlines future research directions.

2. Previous Work

Recent advancements in Android malware detection have shifted towards image-based techniques, where application files are transformed into visual representations for deep learning models [12,13]. These methods leverage computer vision to extract spatial patterns in code, making detection more resilient to obfuscation, encryption, and polymorphic behavior. However, a key distinction in these approaches lies in the choice of image encoding—grayscale versus RGB—which significantly affects model complexity, training efficiency, and generalizability.

To structure the review, Section 2.1 presents prior works based on grayscale image representations, while Section 2.2 focuses on RGB-based methods. Each subsection compares encoding strategies, classification performance, and dataset choices across recent studies.

2.1. Grayscale Representations

Grayscale images offer a single-channel representation of raw bytecode or engineered features, making them lightweight and less memory-intensive.

Casolare et al. (2022) [14] evaluated shallow classifiers trained on both grayscale and RGB images derived from .dex files. While training accuracy reached up to 94.3%, testing accuracy dropped to 85.7%, particularly in RGB cases—highlighting limited generalizability.

Daoudi et al. (2024) [15] proposed DexRay, converting DEX bytecode into grayscale vectors while preserving sequential code order. Unlike standard 2D CNNs that ignore byte ordering, DexRay maintained the instruction sequence, achieving an F1-score of 0.96 on a dataset of 158,000 Android apps.

Mercaldo et al. (2024) [16] combined grayscale and RGB images generated from both static and dynamic features in a GAN-based setup. While classifiers trained on synthetic samples reached an F1-score of 0.8, adversarial examples still evaded detection, underscoring vulnerabilities.

Bakır (2025) [17] introduced TuneDroid, a CNN-based framework that uses grayscale DEX images and employs Bayesian optimization to dynamically tune CNN configurations. TuneDroid achieved 99.44% validation and 98.00% test accuracy, outperforming untuned CNNs and static analysis baselines.

Tang et al. (2025) [18] used grayscale images to represent runtime behavior and network traffic features in DTDroid, achieving 95.14% accuracy. Their approach reinforces the feasibility of grayscale representations in dynamic settings.

El Youssefi (2025) [19] developed an ensemble of six CNNs—ResNet50, EfficientNetB0, AlexNet, DenseNet121, ShuffleNetV2, and MobileNetV2—trained on grayscale images from the CICMalDroid 2020 dataset. The model, optimized with Bayesian fusion, achieved 99.3% accuracy.

In follow-up work, El Youssefi and Chougali (2025) [20] converted multiple APK components (classes.dex, AndroidManifest.xml, resources.arsc) into grayscale images. Their CNN-based fusion model achieved 98.83% accuracy, 99.27% precision, and 99.15% recall, validating the efficacy of multi-source grayscale input.

2.2. RGB Representations

RGB encoding transforms bytecode or feature vectors into three-channel images, enabling models to exploit spatial and color-based patterns. This allows transfer learning from large-scale visual datasets but increases computational cost.

Vasan et al. (2020) [21] used ImageNet-pretrained CNNs fine-tuned on color malware images, reaching 98.82% accuracy on the Maling dataset and 97.35% on the IoT-Android dataset.

Yadav et al. (2022) [22] introduced a DEX-to-RGB conversion using six-digit hexadecimal encoding split into red, green, and blue segments. Classified with EfficientNet-B4, the model achieved 95.7% accuracy, outperforming 26 CNN baselines.

In a related study, Yadav et al. (2022) [23] proposed a two-stage framework combining EfficientNetB0 feature extraction with a stacking ensemble of SVM, Random Forest, and Logistic Regression. The method achieved 100% binary, 92.9% five-class, and 88.6% four-class classification accuracy.

Naeem et al. (2022) [24] combined RGB encoding with Explainable AI tools. Their Inception-v3-based model used Grad-CAM and t-SNE to visualize malware-relevant regions, achieving 98.5% accuracy in binary classification and 91% in multi-class tasks.

Tasyurek and Arslan (2023) [25] introduced RT-Droid, a real-time Android malware detection system that converts AndroidManifest.xml features into RGB images. Using YOLOv5 and transfer learning, the model achieved 98.3% precision and 97.0% F-score, processing each app in just 0.019 s while requiring significantly less memory than grayscale-based approaches.

Ksibi et al. (2025) [26] transformed tabular Android malware data into RGB images through normalization and grid mapping, then classified them using VGG16, achieving 99.25% accuracy on CIC-AndMal2017.

Li et al. (2025) [27] proposed a multimodal architecture fusing source code features with RGB-transformed binaries. Their method reached 98.28% accuracy and 98.66% F1-score by leveraging code semantics alongside visual patterns.

Wasif et al. (2025) [28] explored dynamic feature representation using RGB-encoded network traffic images. Their hybrid CNN-ViT framework achieved 99.61% accuracy in multi-class classification, showcasing the potential of transformer architectures with RGB input.

While prior work (see Table 1) demonstrates strong accuracy using either grayscale or RGB encodings, most approaches fall short in meeting three critical requirements simultaneously: real-time processing, interpretability, and generalizability under code obfuscation. Grayscale-based models tend to offer faster inference and lower memory usage but often suffer from limited feature richness. In contrast, RGB-based methods leverage richer representations—enabling higher classification performance—but typically involve deeper architectures and lack interpretability. Moreover, only a few studies incorporate explainability tools or fuse information from multiple APK components. Our proposed YoloMal-XAI framework directly addresses these gaps by integrating YOLO11 for real-time detection, RGB-based multi-channel encoding for enhanced feature expressiveness, and Explainable AI (XAI) mechanisms for model transparency. This combination enables an efficient, interpretable, and robust Android malware detection system suitable for deployment in dynamic and adversarial environments.

Table 1. Comparison of Android malware detection approaches.

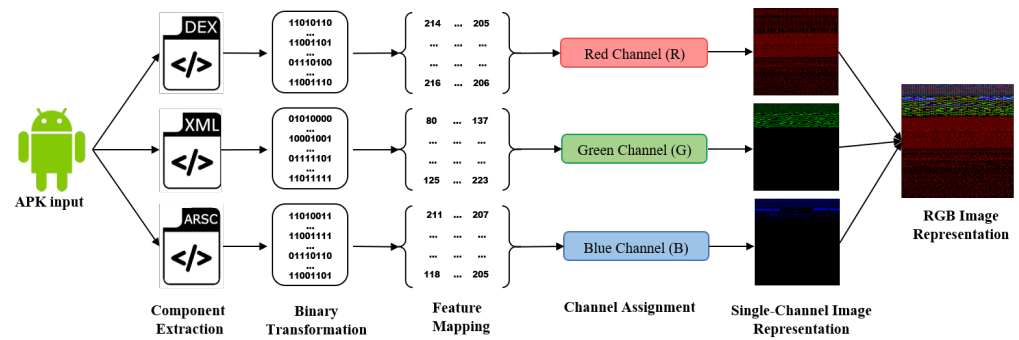
Ref.	Approach	Features	Dataset	Accuracy (%)
[15]	1D-CNN	Bytecode	AndroZoo	97.00
[17]	TuneDroid (Bayesian CNN tuning)	Grayscale DEX images	Custom (6000 apps)	99.44 (val), 98.00 (test)
[18]	DTDroid (Deep DL model)	Grayscale runtime network behavior	Custom	95.14
[19]	6 CNN Ensemble + Bayesian Fusion	Grayscale DEX images	CICMalDroid2020	99.30
[20]	ResNet50 + multi-component grayscale fusion	DEX, Manifest.xml, Resources.arsc	CICMalDroid2020	98.83 (acc), 99.27 (prec), 99.15 (rec)
[21]	IMCFN (ImageNet-based CNN)	Color malware images	Malimg, IoT-Android	98.82 (Malimg), 97.35 (IoT)
[22]	EfficientNet-B4	DEX bytecode RGB	R2-D2	95.70
[23]	EfficientNetB0 + SVM/RF/LogReg	RGB malware images from DEX files	Custom (7312 apps)	100 (binary), 92.9 (5-class), 88.6 (4-class)
[24]	Inception-v3 + Grad-CAM	DEX bytecode RGB	R2-D2, MalNet	98.5 (binary), 91.0 (multi)
[25]	YOLOv5 (RT-Droid)	AndroidManifest.xml (QR-like RGB)	Drebin, Genome, Arslan	98.3 (prec), 97.0 (F1)
[26]	VGG16, VGG19	Network traffic	CIC-AndMal2017	99.25
[27]	Multi-modal fusion with self-attention	Sensitive API, Binary Code	VirusShare, Google Play	98.28
[28]	Hybrid CNN-ViT (2 × CNN + 2 × ViT)	Network traffic RGB	CIC-AndMal2017	99.61
Ours	YOLO11x-cls, YOLO11n-cls	DEX, Manifest.xml, Resources.arsc	CICMalDroid2020	99.87 (binary), 99.56 (multi)

3. Methodology

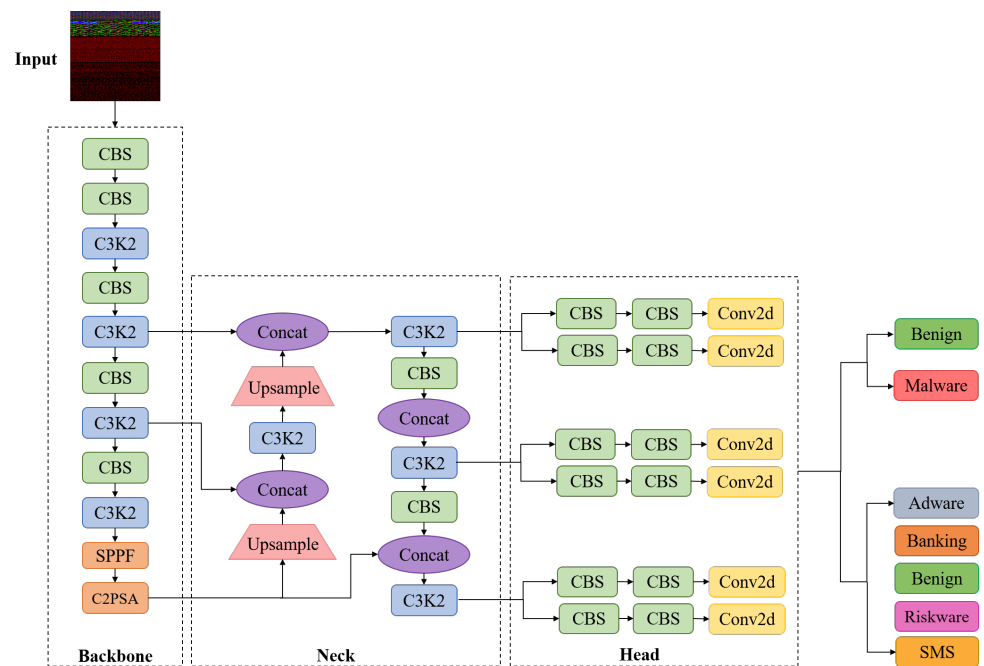
This section presents the YoloMal-XAI framework, a novel approach for Android malware classification that introduces three core innovations. First, we propose a multi-channel RGB encoding strategy that maps the DEX, XML, and ARSC components of an APK to the red, green, and blue channels, respectively, preserving structural semantics while enhancing resilience against obfuscation. Second, we adapt YOLO11—a high-speed convolutional architecture—for both binary and multi-class malware image classification. Third, we incorporate a suite of Explainable AI (XAI) methods, including Grad-CAM, Grad-CAM++, Eigen-CAM, and HiRes-CAM, enabling interpretable predictions through visual and quantitative explanation metrics.

A high-level workflow of the complete pipeline—including APK feature extraction, RGB image construction, YOLO11-based classification, and XAI interpretation—is shown in Figure 1.

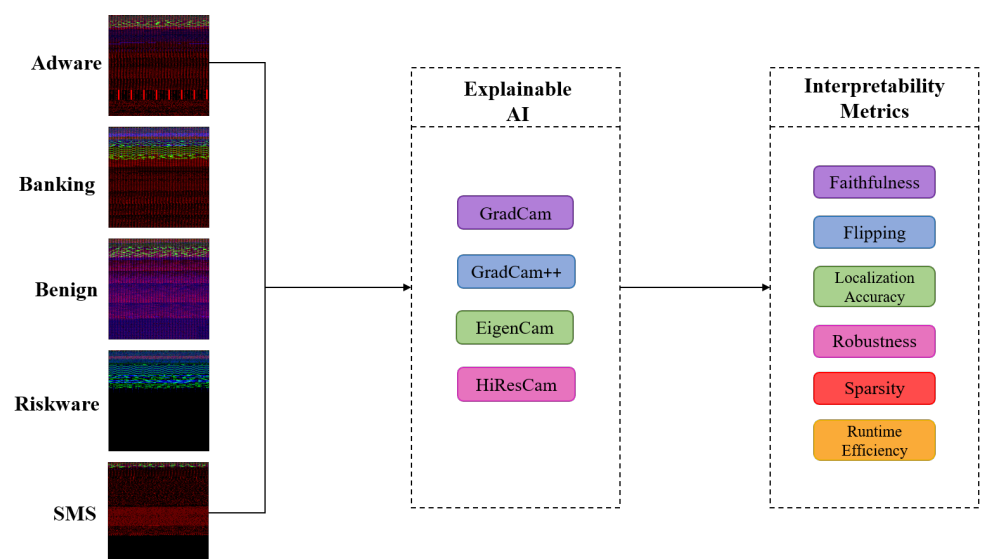
This section is organized as follows. Section 3.1 provides a general overview of the framework. Section 3.2 describes the data preprocessing and RGB image construction. Section 3.3 explains the YOLO11-based malware classification architecture. Section 3.4 discusses the integration of Explainable AI techniques and interpretability evaluation.



(a) APK feature extraction and RGB image representation.



(b) YOLO11-based malware classification.



(c) Explainable AI and interpretability metrics.

Figure 1. YOLO11-based malware detection with XAI architecture. This figure includes: (a) feature extraction from APK files and their transformation into RGB images, (b) malware classification using YOLO11, and (c) Explainable AI interpretability insights.

3.1. Overview of YoloMal-XAI

YoloMal-XAI is a deep learning framework designed to transform APK files into interpretable RGB images and classify them using a modified YOLO11 architecture. The process involves three key stages: (i) feature extraction and RGB image construction from the DEX, Manifest, and Resources components, (ii) image classification using YOLO11, and (iii) explanation generation through XAI methods. Each component is optimized for both performance and interpretability, forming a unified pipeline for accurate and explainable malware detection.

- **APK Feature Extraction and RGB Image Construction:** Three essential components are extracted from APK files: DEX (Dalvik Executable), XML (Manifest), and ARSC (Resources). These components are mapped into an RGB image representation, where DEX, Manifest, and Resources are assigned to the red, green, and blue channels, respectively. This approach preserves the structural and semantic features of the APK while enhancing resilience against obfuscation.
- **YOLO11-Based Classification:** The generated RGB images are fed into the YOLO11 architecture, which consists of a Backbone for feature extraction, a Neck for multi-scale feature fusion, and a Classification Head for final predictions. The model performs both binary classification (Malware vs. Benign) and multi-class classification (Adware, Banking, Riskware, SMS, Benign), achieving high accuracy and real-time performance.
- **Explainable AI (XAI) and Interpretability:** To ensure transparency and trust, we integrate Grad-CAM-based visualization techniques (Grad-CAM, Grad-CAM++, Eigen-CAM, and HiRes-CAM). We further evaluate interpretability using metrics such as faithfulness, localization accuracy, robustness, and sparsity, ensuring the model's explanations are both accurate and reliable.

To reinforce understanding, Table 2 summarizes the three main stages of the YoloMal-XAI framework, including their processes and key outputs.

Table 2. Summary of YoloMal-XAI framework stages.

Stage	Description	Main Output
Feature Extraction	Extract DEX, Manifest, and Resources components from APK files and map them to RGB channels	RGB image that encodes structural and semantic information
Classification	Apply YOLO11 architecture with multi-scale feature fusion for malware prediction	Real-time binary and multi-class prediction
Interpretability	Generate post hoc visual explanations using Grad-CAM, HiRes-CAM, and other CAM-based methods	Heatmaps showing discriminative regions; interpretability scores (faithfulness, robustness, localization, sparsity)

3.2. Data Preprocessing

3.2.1. APK Feature Extraction

Android applications are packaged as APK (Android Package) files, which are compressed archives containing various elements essential for app execution. For malware analysis, we extract three critical files from each APK, as illustrated in Figure 1a:

- **Dalvik Executable (classes.dex):** Contains compiled bytecode executed by the Android Runtime (ART) or Dalvik Virtual Machine (DVM), representing the app's functional logic.

- Manifest (`AndroidManifest.xml`): Declares permissions, components (activities, services), and interactions with the Android system. Malware often exploits this file to request sensitive privileges or hide background services.
- Resources (`resources.arsc`): Stores compiled UI elements, strings, and themes. Malicious applications may modify these to carry out phishing or deceive users.

These files were selected due to their documented significance in Android malware behavior. The manifest's permissions are widely recognized as strong static indicators [29], while UI resources are often manipulated for deceptive intent [30]. Native libraries (`.so`) and assets were excluded: the former are platform-dependent, frequently obfuscated, and inconsistent across samples [31]; the latter primarily contain non-functional media that introduce noise. Including them reduced model performance in preliminary tests.

3.2.2. Binary Transformation

The extracted files are transformed into grayscale matrices suitable for image-based analysis. Each file is read as raw bytes, converted into a hexadecimal string, split into 8-bit segments (0–255), and mapped to pixel intensities. To ensure consistent input dimensions, each component is resized to a fixed resolution of 256×256 pixels.

This resolution strikes a practical balance: it is large enough to preserve discriminative byte-level patterns from varying file sizes, yet small enough to keep the model lightweight and training efficient. Moreover, 256×256 is a common resolution in deep learning pipelines, facilitating compatibility with pre-trained convolutional backbones.

To validate this choice empirically, we evaluated performance using three image sizes: 64×64 , 128×128 , and 256×256 . The 256×256 resolution achieved the highest F1-score and precision while maintaining comparable training time (see Section 4.2). These findings support the use of 256×256 as a standardized resolution throughout the image generation process.

Additionally, we analyzed the file size distributions of the extracted components. DEX files ranged from 0.29 KB to 35,468.52 KB (mean: 1897.40 KB), Manifest files from 0.00 to 2298.60 KB (mean: 1.70 KB), and Resources.arsc files from 0.55 to 33,297.56 KB (mean: 262.27 KB). This heterogeneity reinforces the need for a fixed-size format, normalized through zero-padding or truncation.

3.2.3. RGB Image Representation

To encode structural semantics, we mapped the three extracted files into RGB channels: DEX $\rightarrow$ Red; Manifest $\rightarrow$ Green; and Resources $\rightarrow$ Blue. This enabled the model to learn inter-component relationships through a unified visual representation. During interpretability analysis, highlighted regions in each channel revealed which component contributed most to the prediction.

To ensure the integrity of inputs for image construction, we filtered out APKs that were corrupted, malformed, or missing essential components. To maintain consistency across devices, we also excluded ZIP-level metadata—such as alignment bytes, digital signatures, and compression headers—by reading raw byte streams directly from each target file. Only valid and structurally complete samples were retained for the image generation pipeline.

Although RGB images were stored using 8-bit pixel values in the range $[0, 255]$, normalization was handled internally by the YOLO11 training framework. Images were globally scaled to $[0, 1]$ during preprocessing, without per-channel or dataset-wide standardization. This approach preserves inter-channel contrast while ensuring compatibility with pretrained weights.

The overall process is summarized in Algorithm 1.

Algorithm 1 Byte-to-RGB Image Conversion from APK Components

Require: APK file containing `classes.dex`, `AndroidManifest.xml`, `resources.arsc`

Ensure: Fixed-size RGB image of shape $256 \times 256 \times 3$

- 1: Extract `.dex`, `.xml`, `.arsc` from APK
 - 2: **for** each file in {DEX, Manifest, Resources} **do**
 - 3: Read raw binary
 - 4: Convert to hexadecimal string
 - 5: Split into 2-character hex segments
 - 6: Convert to integers in $[0, 255]$
 - 7: Pad or truncate to 65,536 values
 - 8: Reshape to 256×256 matrix
 - 9: **end for**
 - 10: Assign matrices to R, G, B channels respectively
 - 11: Stack to form final RGB image
 - 12: Save to disk
 - 13: **return** RGB image
-

3.3. YOLO11-Based Classification

YOLO11 is a state-of-the-art deep learning model designed for high-speed image classification and object detection [32]. It introduces several architectural enhancements, including the C3K2 block, Spatial Pyramid Pooling-Fast (SPPF), and Cross-Stage Partial with Spatial Attention (C2PSA), which collectively improve feature extraction, computational efficiency, and classification accuracy [33].

We adapt YOLO11, originally designed for object classification, to analyze Android malware by leveraging its ability to detect patterns in structured data, thereby enabling accurate classification of malicious behavior. Previous studies have demonstrated the effectiveness of YOLO architectures in malware detection using image-based approaches. For instance, RT-Droid [25] employed YOLOv5 for real-time Android malware detection by converting APK features into RGB images. Similarly, an IoT-focused framework [34] utilized YOLOv7 to extract key features from malware images, achieving high classification accuracy.

YOLO11 introduces multiple classification models, each designed to balance speed and accuracy. The available variants—YOLO11n, YOLO11s, YOLO11m, YOLO11l, and YOLO11x—differ in model complexity, parameter count, and computational efficiency. Smaller models, such as YOLO11n and YOLO11s, achieve faster inference speeds with lower computational requirements, making them suitable for real-time applications and deployment on resource-constrained systems. YOLO11m serves as a mid-range variant, offering a balance between computational cost and classification accuracy. In contrast, larger variants, including YOLO11l and YOLO11x, provide higher classification accuracy at the cost of increased inference time and computational demand, enabling them to capture more complex patterns in malware images. These differences are documented in the official YOLO11 benchmark results [35].

Table 3 summarizes the key differences among YOLO11 variants, including their accuracy, parameter count, computational complexity (FLOPs), and expected inference times.

The YOLO11 architecture comprises three main components—Backbone, Neck, and Head—each playing a pivotal role in feature extraction, fusion, and classification, as illustrated in Figure 1b. The Backbone is responsible for extracting hierarchical features from RGB malware representations and forms the foundation of the model. It employs

Convolution–BatchNorm–SiLU (CBS) layers throughout to stabilize features and incorporate non-linearity using the SiLU activation function. In addition, the Backbone utilizes C3K2 blocks, which efficiently replace large kernels with two smaller convolutions, thereby reducing computational overhead without sacrificing feature quality. To further enhance multi-scale representation learning, a Spatial Pyramid Pooling-Fast (SPPF) module aggregates features across different spatial resolutions, and a Cross-Stage Partial with Spatial Attention (C2PSA) layer at the final stage boosts spatial attention, refining the extracted features before passing them to the next stage.

Table 3. Comparison of YOLO11 variants for classification. Metrics are reported at 224×224 resolution. Inference time is measured using TensorRT on NVIDIA T4 GPU. Adapted from [35].

Model	Top-1 Accuracy (%)	Parameters (M)	FLOPs (B)	Inference Time (ms)
YOLO11n-cls	70.0	1.6	0.5	1.1
YOLO11s-cls	75.4	5.5	1.6	1.3
YOLO11m-cls	77.3	10.4	5.0	2.0
YOLO11l-cls	78.3	12.9	6.2	2.8
YOLO11x-cls	79.5	28.4	13.7	3.8

The Neck integrates and fuses features extracted at various scales to improve the detection of intricate malware patterns. It accomplishes this by upsampling features and merging spatial and channel-wise information, thus enriching the multi-scale feature representations. The integration process is further refined using additional C3K2 blocks, and the continued application of CBS layers helps generalize the features, ensuring that the subsequent classification stage receives robust inputs.

Finally, the Head processes the refined features from the Neck to generate the final malware classification. This stage further refines the feature maps using CBS layers and then employs Conv2D layers to map these processed features to specific malware categories. A Softmax activation function is used to compute the class probabilities, enabling the model to handle both binary classification (benign vs. malware) and multi-class tasks (e.g., distinguishing between adware, banking malware, etc.).

3.4. Explainable AI for Malware Classification

Deep learning models, including YOLO11, operate as black-box systems, making it challenging to interpret their decision-making process. Explainable AI (XAI) techniques address this issue by providing insights into how the model identifies malware patterns from APK-derived RGB images. In this study, we utilize several XAI tools and evaluation metrics to analyze the model's predictions, as illustrated in Figure 1c.

3.4.1. XAI Tools

To enhance interpretability, we employ the following visualization techniques:

- Grad-CAM (Gradient-weighted Class Activation Mapping): Highlights the most influential regions in the image by analyzing the gradient information of the final convolutional layer.
- Grad-CAM++: An extension of Grad-CAM that provides finer localization of important image features by considering pixel-wise importance.
- Eigen-CAM: Uses principal component analysis (PCA) on feature maps to highlight the most dominant patterns in the image.
- HiRes-CAM: A high-resolution version of Grad-CAM that captures fine-grained details in the malware image, improving feature attribution precision.

To generate heatmap-based explanations for Android malware classification, we employed the open-source pytorch-grad-cam library (MIT license, available at [36]), which provides modular implementations of popular Class Activation Mapping (CAM) techniques. We used version $\geq 1.4.0$ without internal modifications. Four CAM-based XAI methods were integrated into our pipeline: Grad-CAM, Grad-CAM++, Eigen-CAM, and HiRes-CAM. These methods were applied to the final convolutional layer of the YOLO11 model (i.e., $\text{layer} = -2$) within the Ultralytics framework. All RGB malware images were resized to 256×256 using a letterbox function and normalized to the $[0, 1]$ range. Heatmaps were computed using backpropagated gradients and overlaid on the original images to highlight regions that contributed most to the classification.

The overall architecture of this CAM-based visualization pipeline is illustrated in Figure 2, which demonstrates the sequential steps from input image preprocessing to the generation of class-discriminative heatmaps using four different CAM techniques.

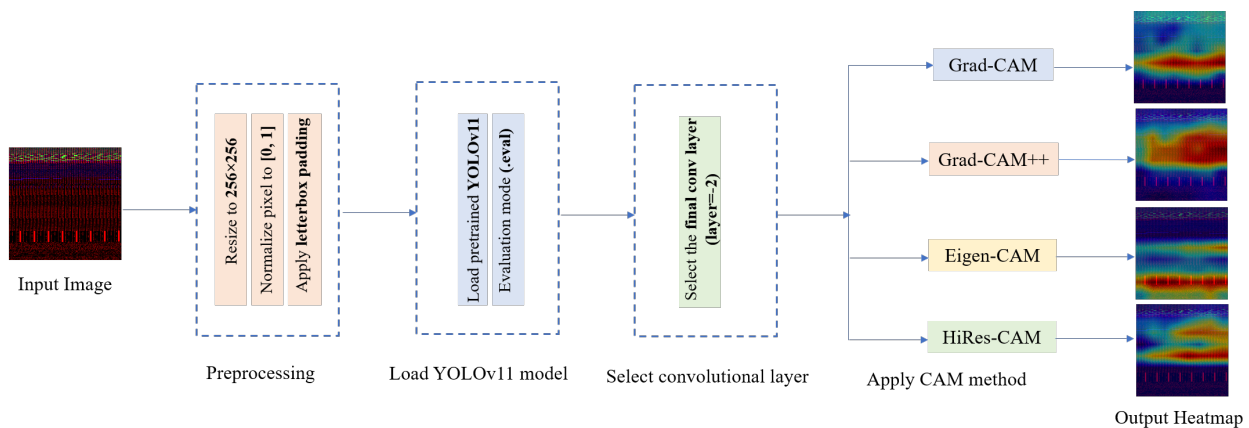


Figure 2. XAI pipeline in YoloMal-XAI. The input image is preprocessed and passed through YOLO11, with four CAM methods applied to the final convolutional layer to generate class-relevant heatmaps.

The entire process is further detailed in Algorithm 2, which outlines the core steps of the explainability routine.

Algorithm 2 CAM-Based Explainability for YOLO11 Classification

Require: Trained YOLO11 model and malware RGB input image

Ensure: RGB heatmap highlighting relevant regions

- 1: Load YOLO11 model and set to evaluation mode
 - 2: Resize input image to 256×256 using letterbox; normalize to $[0, 1]$
 - 3: Select final convolutional layer (e.g., $\text{layer} = -2$)
 - 4: Choose CAM method: Grad-CAM, Grad-CAM++, Eigen-CAM, or HiRes-CAM
 - 5: Forward image through model and compute activation maps using selected CAM method
 - 6: Generate RGB heatmap highlighting class-relevant regions
 - 7: **return** Visual explanation of model decision
-

While CAM-based visualizations enhance interpretability, they also have limitations in a security context. Heatmaps may be sensitive to adversarial perturbations, where slight modifications to an APK's structure can significantly alter the attribution output without changing the classification result. Moreover, obfuscated or packed malware samples may produce diffuse or misleading heatmaps, reducing the reliability of visual interpretations [37]. These limitations should be considered when deploying XAI tools in high-stakes malware analysis pipelines.

3.4.2. Interpretability Metrics Calculation

To quantitatively assess the effectiveness of the explainability techniques applied to YOLO11-based malware classification, several interpretability metrics are computed. These metrics evaluate how well the explainability methods highlight critical malware patterns in the RGB image representations.

- **Faithfulness:** Measures how well the heatmap reflects the model's decision-making process. It is computed by correlating the intensity of heatmap activations with the predicted confidence score:

$$F = \frac{1}{N} \sum_{i=1}^N H(i)P(i) \quad (1)$$

where $H(i)$ is the heatmap activation at pixel i , and $P(i)$ is the prediction confidence at the corresponding location. A higher F value indicates a stronger correlation between the heatmap and the model's prediction.

- **Flipping:** Assesses the stability of the explanation by removing highly activated regions from the image and analyzing how much the model's prediction changes. It is defined as

$$\Delta P = |P_{original} - P_{perturbed}| \quad (2)$$

where $P_{original}$ is the original classification confidence and $P_{perturbed}$ is the confidence after masking out the most activated regions. A lower ΔP means the explanation is more stable.

- **Localization Accuracy:** Evaluates whether the heatmap correctly highlights the region containing malware-relevant features. This is measured using the Intersection over Union IoU :

$$IoU = \frac{|H \cap G|}{|H \cup G|} \quad (3)$$

where H is the binarized heatmap, and G is the ground-truth region. Since pixel-level malware annotations are unavailable in static RGB-encoded APK representations, we synthesized G heuristically. Specifically, we assumed that the most informative patterns originate from the DEX component (mapped to the red channel), and defined G as the top 20% most activated pixels in that region. This provides a reproducible, albeit approximate, proxy for evaluating spatial alignment between heatmap activations and malware-relevant regions. A higher IoU score indicates that the explanation method successfully localizes critical areas in the input image.

- **Robustness:** Quantifies how stable the heatmap remains under small perturbations in the input image. It is computed using the Structural Similarity Index Measure $SSIM$:

$$SSIM(H, H') = \frac{(2\mu_H\mu_{H'} + C_1)(2\sigma_{HH'} + C_2)}{(\mu_H^2 + \mu_{H'}^2 + C_1)(\sigma_H^2 + \sigma_{H'}^2 + C_2)} \quad (4)$$

where H is the original heatmap, H' is the perturbed heatmap, μ and σ are their means and standard deviations, and C_1, C_2 are small constants to prevent division by zero. A higher $SSIM$ indicates better robustness.

- **Sparsity:** Measures how compact the heatmap explanation is, favoring methods that highlight only the most relevant regions. It is calculated as

$$S = \frac{|H > 0|}{|H|} \quad (5)$$

where $|H > 0|$ is the number of activated pixels, and $|H|$ is the total number of pixels. A lower sparsity value indicates a more concentrated and meaningful explanation.

- **Runtime Efficiency:** Measures the computational cost of generating heatmaps using different explainability techniques. It is computed as

$$R = t_{end} - t_{start} \quad (6)$$

where t_{start} and t_{end} are the timestamps recorded before and after generating the heatmap. A lower R value indicates faster execution.

While insertion and deletion metrics are commonly used in interpretability studies, we opted for a broader set of metrics—faithfulness, flipping, IoU, SSIM, sparsity, and runtime—to better capture multiple aspects of explanation quality in a security-oriented context. Insertion/deletion methods simulate gradual occlusion or information gain but are computationally expensive and less interpretable in RGB malware images, where pixel-level semantics are abstract. In contrast, our chosen metrics offer a balance between computational efficiency and the ability to assess attribution relevance (faithfulness, flipping), spatial localization (*IoU*), visual stability (*SSIM*), conciseness (sparsity), and real-world usability (runtime). This multidimensional evaluation provides a more comprehensive understanding of XAI method performance in malware analysis pipelines.

4. Evaluation and Results

This section presents a comprehensive evaluation of the proposed YoloMal-XAI framework as follows. Section 4.1 introduces the dataset and the preprocessing pipeline. Section 4.2 presents the experimental setup and training hyperparameters. Section 4.3 defines the evaluation metrics used to assess classification performance. Section 4.4 reports the results for both binary and multi-class classification using different YOLO11 variants. Section 4.5 compares YoloMal-XAI with CNN baselines and YOLOv8. Section 4.6 presents an ablation study analyzing the contribution of different APK components. Finally, Section 4.7 evaluates model generalization through cross-corpus validation using external datasets.

4.1. Dataset

In this study, we utilize the CICMalDroid 2020 dataset [38], a benchmark dataset for Android malware detection. To ensure the reliability of the dataset, we performed a detailed quality screening phase prior to image conversion and analysis. A total of 993 APKs (5.9% of the original set) were excluded due to structural or extraction-related issues. Specifically, 748 APKs failed ZIP archive validation, indicating corruption or misformatting. Additionally, 31 APKs lacked the essential `.dex` component, and 3 were missing `resources.arsc`. The remaining samples were filtered out due to decompression errors (e.g., invalid block types or bad CRC-32 checksums) and unreadable Unicode file paths causing I/O exceptions. These exclusions reduced noise and ensured that only structurally complete and analyzable APKs were retained, resulting in a high-quality corpus of 15,849 samples.

While CICMalDroid2020 provides a diverse set of malware families, it exhibits notable class imbalance. For example, SMS malware (4505 samples) significantly outnumbers adware (1469), which may bias the model toward overrepresented categories. To mitigate this, we performed an 80/20 stratified split during train–test partitioning to preserve class proportions. Additionally, we reported class-wise metrics in both binary and multi-class classification settings to ensure balanced evaluation despite the skewed distribution.

To evaluate the generalizability of our approach beyond CICMalDroid2020, we performed cross-dataset validation using two external corpora: the Drebin dataset [39]

(500 malware APKs labeled via VirusTotal) and CICAndMal2017 [40] (500 benign APKs with verified ground truth).

4.2. Hyperparameters and Experimental Setup

To train the YOLO11-based malware classification models effectively, we performed a structured hyperparameter tuning process. We adopted a stepwise manual tuning strategy. This allowed us to isolate and evaluate the effect of each core parameter—learning rate, input image size, and optimizer—while keeping the other settings fixed. All tuning experiments were performed using the lightweight YOLO11n variant under a binary classification setting to enable fast iterations and controlled comparisons.

We first evaluated the impact of learning rate (lr0) by testing four values: 0.1, 0.01, 0.001, and 0.0001. As shown in Table 4, the model trained with 0.001 achieved a strong trade-off between accuracy (99.68%) and training time (48 min). Although 0.0001 slightly improved the F1-score, it incurred a longer training time (1 h 6 min). Based on this, we selected 0.001 as the default for all subsequent experiments.

Table 4. Performance comparison for different learning rates.

Learning Rate	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)
0.1	96.41	97.88	97.30	97.58	1.10
0.01	99.27	99.16	99.87	99.52	1.15
0.001	99.68	99.87	99.70	99.79	0.81
0.0001	99.78	99.83	99.87	99.85	1.09

Next, we compared three input resolutions: 64×64 , 128×128 , and 256×256 . The 256×256 configuration achieved the best F1-score (99.79%) and maintained efficient training time (0.81 h), as shown in Table 5. This resolution was adopted as the default setting for all models.

Table 5. Performance comparison of different input image sizes

Image Size	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)
64×64	99.59	99.70	99.75	99.73	0.77
128×128	99.56	99.75	99.66	99.70	0.80
256×256	99.68	99.87	99.70	99.79	0.81

Finally, we tested three optimizers: Adam, AdamW, and SGD. While SGD produced the highest F1-score (99.83%), it required significantly more training time. Adam was faster but less accurate. AdamW delivered strong overall performance with the fastest training time (0.81 h), making it the most efficient and robust choice for our task. We therefore selected AdamW for all YOLO11 model variants. The results of this evaluation are summarized in Table 6.

AdamW extends the standard Adam optimizer by decoupling weight decay from the gradient update, which improves generalization and helps prevent overfitting. This decoupling mechanism enhances training stability, particularly in large-scale or imbalanced classification problems [41].

Table 6. Performance comparison of different optimizers.

Optimizer	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)
Adam	99.24	99.66	99.32	99.49	1.18
AdamW	99.68	99.87	99.70	99.79	0.81
SGD	99.75	99.75	99.92	99.83	1.51

To enhance convergence and generalization, we applied a cosine learning rate decay schedule (`cos_lr = True`), warmup epochs (`warmup_epochs = 3`), and a small weight decay of 0.0005. We also used mixup data augmentation (`mixup = 0.2`) and label smoothing was disabled (`label_smoothing = 0.0`) to retain class purity. The complete set of final training configurations is summarized in Table 7, which outlines the hyperparameter values used across all YOLO11 variants in the main experiments.

Table 7. YOLO11 final training configuration.

Parameter	Value
Epochs	100
Input Image Size	256 × 256
Batch Size	32
Optimizer	AdamW
Initial Learning Rate (lr0)	0.001
Weight Decay	0.0005
Cosine Learning Rate Schedule (<code>cos_lr</code>)	Enabled
Warmup Epochs	3
Warmup Momentum	0.8
Mixup Augmentation	0.2
Auto Augmentation	RandAugment
Pretrained Weights	Yes
Model Variants	YOLO11n, YOLO11s, YOLO11m, YOLO11l, YOLO11x

The model is implemented and evaluated on the hardware and software setup detailed in Table 8.

Table 8. Hardware and software setup for model evaluation.

Name	Type
GPU	NVIDIA RTX 3050 Ti
CPU	11th Gen Intel® Core™ i5-11400H
Deep Learning Framework	PyTorch 2.4.1
CUDA Version	11.8
RAM	16 GB
Operating System	Windows 11

4.3. Evaluation Metrics

To evaluate the performance of YoloMal-XAI, we employ several metrics designed to assess both binary and multi-class classification tasks. These metrics help in understanding the model's ability to distinguish between benign and malicious APK samples, as well as to classify the samples accurately across the five malware categories. The following evaluation metrics are employed:

- Accuracy: Measures the proportion of correctly classified samples (both benign and malware) out of all samples.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}} \quad (7)$$

- Precision: Evaluates the proportion of true-positive samples among all predicted positives.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (8)$$

- Recall (Sensitivity): Assesses the ability to identify all true instances of a class.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (9)$$

- F1-Score: The harmonic mean of precision and recall, offering a balanced evaluation for imbalanced datasets.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

- Area Under the ROC Curve (AUC-ROC): For binary classification, we compute the AUC-ROC to evaluate the trade-off between true-positive rate (TPR) and false-positive rate (FPR) across thresholds. A higher AUC indicates better discriminative ability.
- Macro- vs. Micro-Averaged F1 (Multi-Class):
 - Macro-F1 treats all classes equally by computing the unweighted mean F1 across classes.
 - Micro-F1 aggregates true positives, false negatives, and false positives globally before computing the F1-score. This is useful when class imbalance exists.

4.4. Results

We assess YoloMal-XAI's performance across binary and multi-class classification tasks to evaluate its robustness and scalability using all YOLO11 variants. Table 9 presents a unified comparison of classification accuracy, precision, recall, F1-score, and training time for both settings.

In the binary classification task, YOLO11x achieves the best performance overall (99.87% accuracy, 99.92% F1-score), benefiting from its increased capacity, albeit at a high computational cost (6.42 h). YOLO11s and YOLO11m yield identical F1-scores (99.83%), with YOLO11s requiring significantly less training time (1.17 h vs. 2.04 h), making it a more efficient mid-range option. YOLO11n demonstrates excellent performance given its compact architecture, achieving 99.68% accuracy in just 0.81 h—ideal for real-time or resource-constrained deployments.

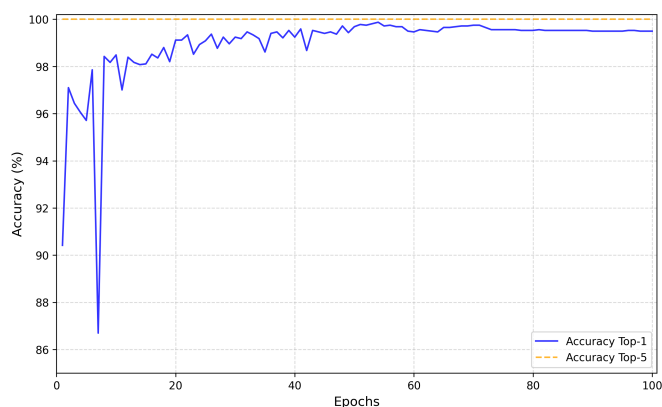
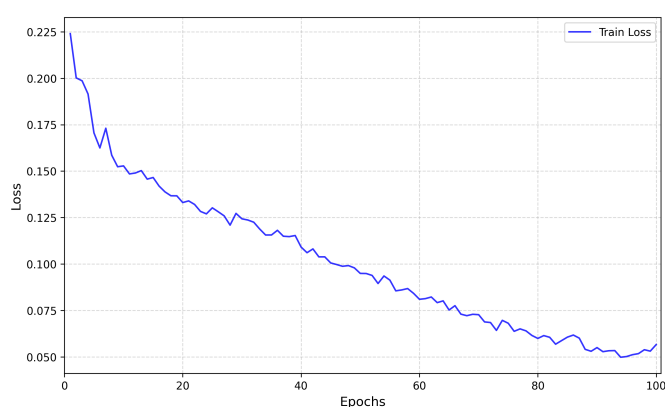
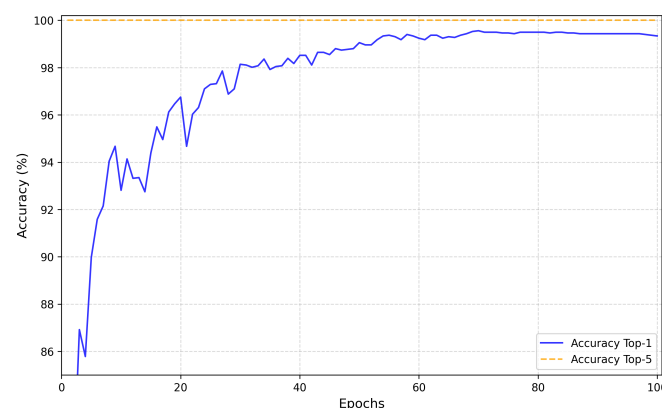
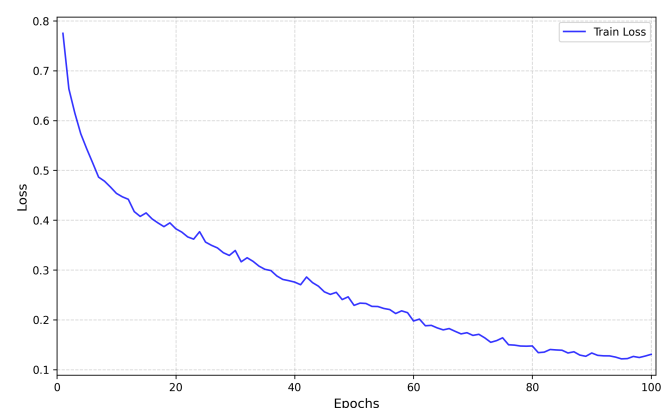
In the multi-class scenario, YOLO11n leads with the highest accuracy (99.56%) and F1-score (99.55%), outperforming deeper models despite its smaller size. Larger variants such as YOLO11x (98.90%) and YOLO11m (99.15%) show a slight drop in performance while incurring longer training times. YOLO11s and YOLO11l offer balanced trade-offs, with accuracy around 99.3–99.4% and moderate training duration.

These results suggest that YOLO11n is better suited for fine-grained multi-class malware categorization, while YOLO11x excels in binary discrimination when computational cost is not a constraint.

Table 9. Comparison of YOLO11 variants on binary and multi-class malware classification.

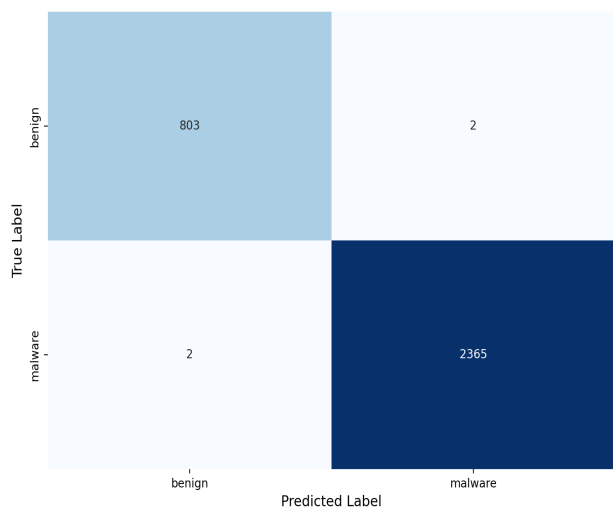
Model	Binary Classification					Multi-Class Classification				
	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)
YOLO11n	99.68	99.87	99.70	99.79	0.81	99.56	99.51	99.58	99.55	1.07
YOLO11s	99.75	99.79	99.87	99.83	1.17	99.43	99.29	99.15	99.22	1.13
YOLO11m	99.75	99.79	99.87	99.83	2.04	99.15	99.17	98.92	99.04	2.06
YOLO11l	99.81	99.96	99.79	99.87	2.71	99.34	99.28	99.04	99.16	2.72
YOLO11x	99.87	99.92	99.92	99.92	6.42	98.90	98.82	98.34	98.57	6.81

We analyze the training dynamics of YoloMal-XAI by visualizing the learning curves of two representative YOLO11 variants. YOLO11x, which achieved the highest binary classification performance, is shown in Figure 3, highlighting both its accuracy and loss over epochs. In contrast, Figure 4 presents the training behavior of YOLO11n, which demonstrated superior generalization in the multi-class setting.

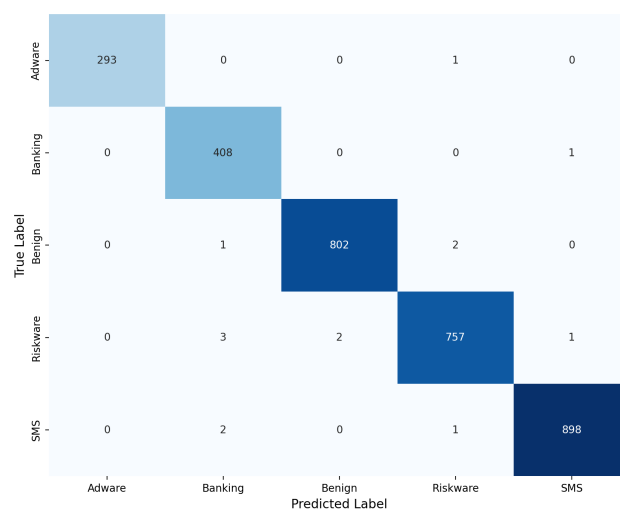
**(a)** YOLO11x accuracy vs. epoch (binary classification).**(b)** YOLO11x loss vs. epoch (binary classification).**Figure 3.** Learning curves for YOLO11x on binary classification: **(a)** accuracy and **(b)** loss.**(a)** YOLO11n accuracy vs. epoch (multi-class classification).**(b)** YOLO11n loss vs. epoch (multi-class classification).**Figure 4.** Learning curves for YOLO11n on multi-class classification: **(a)** accuracy and **(b)** loss.

To evaluate the classification performance in detail, we present the confusion matrices of the best-performing YOLO11 variants in both tasks. Figure 5 shows the binary confusion matrix for YOLO11x and the multi-class confusion matrix for YOLO11n. For binary classification, YOLO11x correctly identified 803 benign and 2365 malware samples, with only 2 false positives and 2 false negatives—highlighting a low false-positive rate, which

is crucial for security applications. In the multi-class case, YOLO11n achieved 3158 correct predictions across the five malware categories, with minor misclassifications such as 3 Riskware instances predicted as Banking and 2 SMS samples misclassified as Banking, indicating slightly overlapping feature representations in these classes.



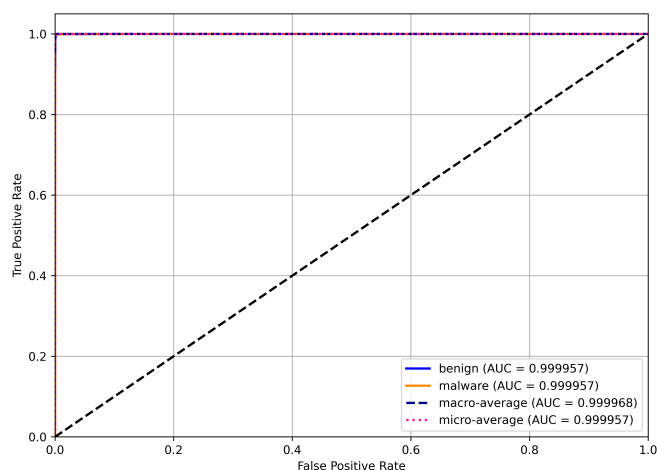
(a) Binary confusion matrix (YOLO11x)



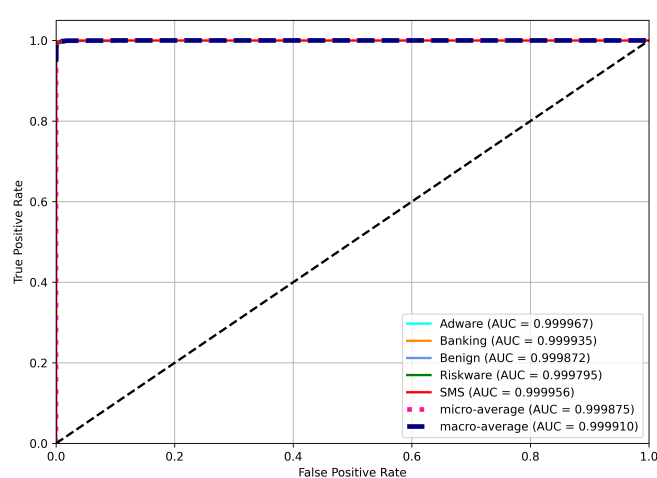
(b) Multi-class confusion matrix (YOLO11n)

Figure 5. Confusion matrices for the best-performing YOLO11 variants: YOLO11x (binary) and YOLO11n (multi-class).

To further assess the discriminative capability of YoloMal-XAI, we analyze the Receiver Operating Characteristic (ROC) curves for both binary and multi-class classification tasks. ROC curves provide a graphical representation of the trade-off between true-positive and false-positive rates across various thresholds, and the Area Under the Curve (AUC) quantifies the model's ability to distinguish between classes. Figure 6a,b illustrate the ROC performance for YOLO11x (binary) and YOLO11n (multi-class), respectively. Both configurations achieve AUC values exceeding 0.999, indicating excellent classification performance. Additionally, micro- and macro-averaged AUCs are reported for the multi-class scenario, demonstrating consistent performance across all malware categories.



(a) ROC curve for YOLO11x (binary classification).



(b) ROC curve for YOLO11n (multi-class classification).

Figure 6. ROC analysis of YoloMal-XAI in (a) binary and (b) multi-class classification settings, showing near-perfect separability across classes.

4.5. Comparison with CNN and YOLOv8 Baselines

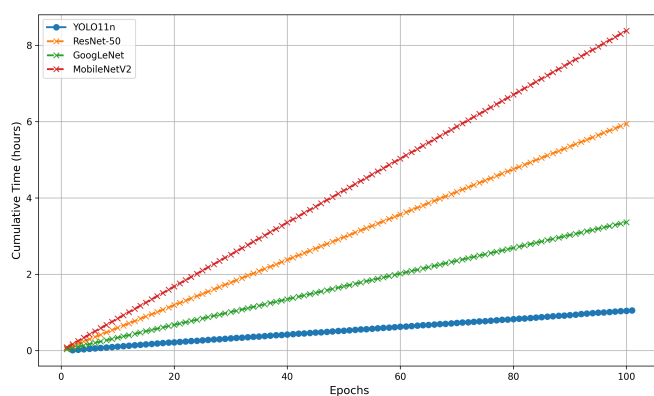
To comprehensively evaluate YoloMal-XAI's performance, we benchmarked the YOLO11n variant against three widely adopted CNN architectures: ResNet-50 (deep and heavyweight), GoogLeNet (mid-sized with Inception modules), and MobileNetV2 (lightweight and mobile-optimized). All models were trained under identical conditions for 100 epochs using the same hyperparameters and datasets.

Table 10 summarizes the binary and multi-class classification results. YOLO11n consistently outperforms all CNN baselines in both accuracy and training efficiency. It achieves 99.68% accuracy in binary classification with a training time of only 0.81 h, and 99.56% accuracy, 99.51% precision, 99.58% recall, and 99.55% F1-score in multi-class classification within 1.07 h. In contrast, ResNet-50, GoogLeNet, and MobileNetV2 exhibit lower accuracy and require significantly more computational time.

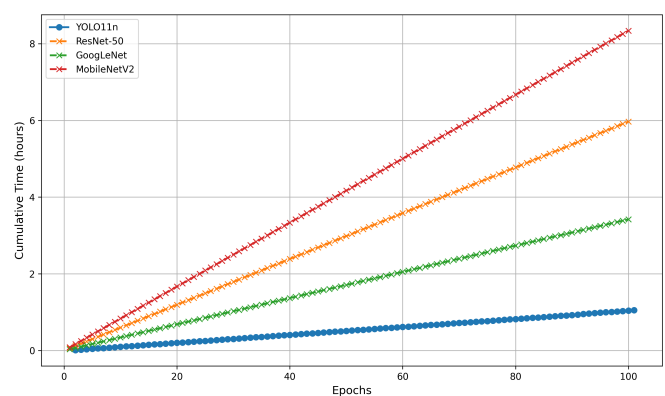
Table 10. Performance comparison of YOLO11n and CNN models on binary and multi-class classification tasks.

Model	Binary Classification					Multi-Class Classification				
	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Time (h)	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Time (h)
ResNet50	99.40	99.45	99.74	99.60	5.94	97.88	97.92	97.88	97.88	5.97
GoogLeNet	99.27	99.57	99.45	99.51	3.36	97.79	97.85	97.93	97.80	3.42
MobileNet	99.24	99.28	99.70	99.49	8.38	97.66	97.71	97.66	97.66	8.34
YOLO11n	99.68	99.87	99.70	99.79	0.81	99.56	99.51	99.58	99.55	1.07

These findings are further illustrated in Figure 7, which shows the learning curves of YOLO11n compared to CNN models. YOLO11n converges faster and more stably across both tasks. With only 1.6M parameters [35], it surpasses MobileNetV2 in both performance and efficiency, while also being more computationally practical than larger models like ResNet-50.



(a) Binary classification: YOLO11n vs. CNN models.



(b) Multi-class classification: YOLO11n vs. CNN models.

Figure 7. Time vs. epoch plots comparing YOLO11n with CNN models for (a) binary and (b) multi-class classification tasks.

In addition to CNN-based baselines, we benchmarked YoloMal-XAI against YOLOv8, a state-of-the-art architecture widely used for image classification. To ensure fairness, we selected comparable configurations from each family: YOLO11x and YOLOv8x for binary classification, and YOLO11n and YOLOv8n for multi-class classification. All models were trained using the same datasets, preprocessing pipeline, and hyperparameters.

As shown in Table 11, YOLO11x achieves a F1-score of 99.92% with a training time of only 6.42 h—significantly faster than YOLOv8x, which required 22.02 h to reach a comparable F1-score of 99.94%. This makes YOLO11x approximately 3.5× more time-efficient.

In the multi-class task, YOLO11n attained 99.56% accuracy and F1-score in just 1.07 h, outperforming YOLOv8n, which reached 98.87% accuracy and 98.86% F1-score in 0.99 h. Despite YOLOv8n’s slightly faster runtime, YOLO11n achieved superior classification metrics.

Table 11. Performance comparison between YOLO11 and YOLOv8 for binary and multi-class classification tasks.

Task	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Training Time (h)
Binary	YOLO11x	99.87	99.92	99.92	99.92	6.42
	YOLOv8x	99.91	99.92	99.96	99.94	22.02
Multi-Class	YOLO11n	99.56	99.56	99.56	99.56	1.07
	YOLOv8n	98.87	98.88	98.87	98.86	0.99

4.6. Ablation Study

To evaluate the contribution of each APK component in the proposed YoloMal-XAI framework, we conducted a comprehensive ablation study across both binary and multi-class classification tasks. Seven configurations were tested: DEX only, Manifest only, Resources only, DEX + Manifest, DEX + Resources, Manifest + Resources, and the full RGB combination incorporating all three. The RGB channel mappings for each configuration are detailed in Table 12, and example image representations are shown in Figure 8, highlighting structural differences across feature combinations.

Table 12. RGB channel mapping for each APK feature combination used in image generation.

Feature Combination	Red (R)	Green (G)	Blue (B)
DEX only	DEX	0	0
Manifest only	0	Manifest	0
Resources only	0	0	Resources
DEX + Manifest	DEX	Manifest	0
DEX + Resources	DEX	0	Resources
Manifest + Resources	0	Manifest	Resources
DEX + Manifest + Resources	DEX	Manifest	Resources

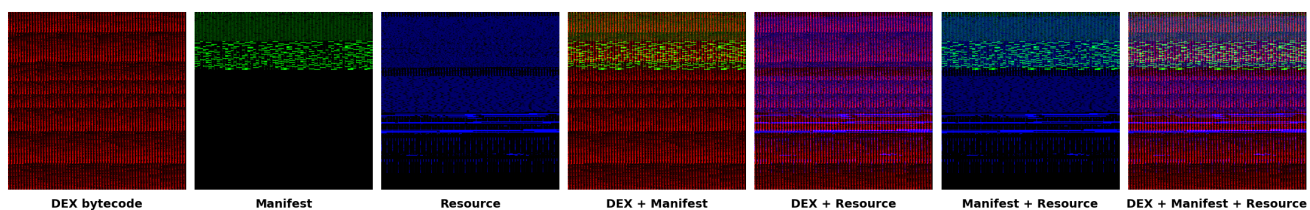


Figure 8. Visual comparison of image encodings based on different APK components: (left to right) DEX only, Manifest only, Resources only, and their pairwise and full combinations. Each RGB configuration reveals unique structural features used by the classifier.

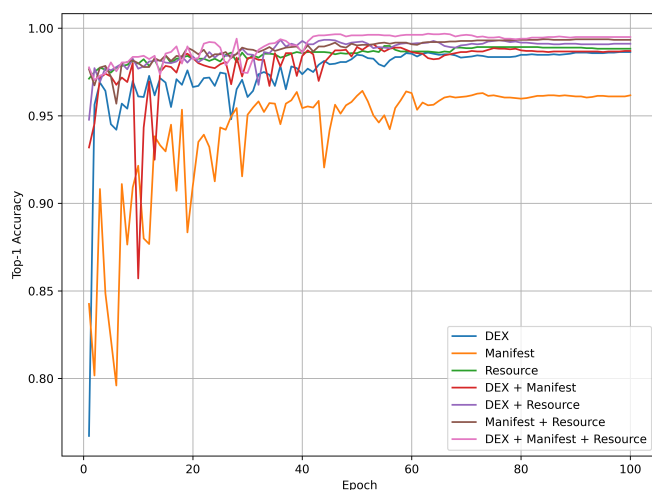
The results, presented in Table 13, reveal that the DEX component alone achieves remarkably high performance, with 98.64% accuracy in binary classification and 96.77% in multi-class classification. This confirms that the Dalvik bytecode remains the primary feature carrying discriminative information for malware detection, as it encodes the executable logic and API calls characteristic of malicious behaviours. In contrast, the Manifest.xml component shows moderate performance, achieving 96.29% binary accuracy and

91.64% multi-class accuracy, indicating that while permission and component declarations provide useful indicators of suspicious activities, they lack detailed behavioural semantics to support precise malware family classification. Resources.arsc alone yields 98.99% binary classification accuracy but only 91.83% in multi-class tasks, suggesting that static resources such as icons, layouts, and strings can reflect repackaging or branding modifications typical in malware repackaging attacks, yet remain insufficient for distinguishing between malware families. Combining DEX with either Manifest or Resources leads to significant performance improvements, demonstrating the complementary nature of semantic and structural cues encoded in these APK components. For instance, the combination of DEX and Resources achieves 99.33% binary and 98.57% multi-class accuracy, while DEX and Manifest achieve 99.08% and 98.07%, respectively.

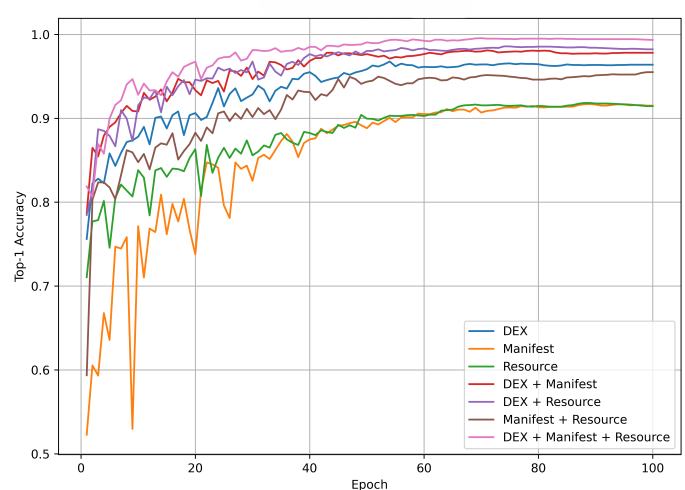
Table 13. Ablation study comparing individual and combined APK features (DEX, Manifest, and Resources) for binary and multi-class malware classification.

Feature Set	Binary Classification					Multi-Class Classification				
	Acc. (%)	Prec. (%)	Rec. (%)	F1-Score (%)	Train. Time (h)	Acc. (%)	Prec. (%)	Rec. (%)	F1-Score (%)	Train. Time (h)
DEX	98.64	98.54	99.66	99.10	1.24	96.77	96.79	96.77	96.78	1.15
Manifest	96.29	96.75	98.35	97.54	1.22	91.64	91.73	91.64	91.66	1.15
Resources	98.99	99.32	99.32	99.32	1.19	91.83	92.09	91.83	91.77	1.15
DEX + Manifest	99.08	99.20	99.58	99.39	1.51	98.07	98.08	98.07	98.07	1.11
DEX + Resources	99.33	99.49	99.62	99.56	1.20	98.57	98.57	98.57	98.57	1.12
Manifest + Resources	99.37	99.62	99.53	99.58	1.13	95.50	95.57	95.50	95.50	1.17
DEX + Manifest + Resources	99.68	99.87	99.70	99.79	0.81	99.56	99.56	99.56	99.56	1.07

To visualize the training behavior across different feature combinations, Figure 9 presents the Top-1 accuracy curves for binary and multi-class classification. The curves confirm that configurations involving the DEX component converge faster and more stably, with the full RGB combination (DEX + Manifest + Resource) consistently achieving the best performance.



(a) Binary classification accuracy curves.



(b) Multi-class classification accuracy curves.

Figure 9. Top-1 accuracy training curves for the ablation study. The full RGB combination consistently outperforms individual and partial feature sets in both binary and multi-class settings.

4.7. Cross-Corpus Validation

To assess the generalization ability of our YOLO11-based malware classifier beyond the CICMalDroid2020 dataset, we conducted a cross-corpus validation experiment using a balanced external set comprising 500 malware samples from the Drebin dataset and 500 benign samples from CICAndMal2017. Rather than retraining, we used the final checkpoint weights (best.pt) of each pre-trained YOLO11 variant, previously fine-tuned on CICMalDroid2020. Each model was evaluated on the 1000-sample test set using the original RGB image representations, and predictions were obtained through direct inference.

Table 14 reports the results, showing that all YOLO11 variants generalize well to unseen APKs from independent sources, with YOLO11x achieving perfect scores (100% across all metrics).

Table 14. Binary classification performance of YOLO11 variants on cross-corpus validation set.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
YOLO11n	99.30	98.62	100.00	99.30
YOLO11s	99.10	98.23	100.00	99.11
YOLO11m	99.00	98.61	99.40	99.00
YOLO11l	99.10	99.60	98.60	99.10
YOLO11x	100.0	100.0	100.0	100.0

For multi-class validation, we merged 1000 APK samples without access to fine-grained malware family labels, and we used the best-performing YOLO11n model (selected based on CICMalDroid2020 results) to perform inference on the combined dataset. The goal was to assess the model's distribution of predictions and confidence scores across known malware families (e.g., SMS, Adware, Riskware, Banking).

As summarized in Table 15, the YOLO11n classifier predicted 515 samples as benign with an average confidence of 97.9%, followed by 197 as SMS (92.3%), 188 as Adware (82.7%), 53 as Riskware (64.8%), and 47 as Banking (61.0%). These predictions indicate that the model can meaningfully differentiate among malware types even in the absence of exact ground-truth labels.

Table 15. Prediction distribution and average confidence of YOLO11n on the merged cross-corpus dataset (Drebin + CICAndMal2017).

Predicted Class	Count	Average Confidence
Benign	515	0.979
SMS	197	0.923
Adware	188	0.827
Riskware	53	0.648
Banking	47	0.610

This cross-corpus validation strategy mitigates dataset bias and confirms the model's robustness against distributional shifts. The strong results across two independent datasets support the applicability of YoloMal-XAI in real-world malware detection scenarios involving diverse and previously unseen APKs.

4.8. Explainable AI Visualization

To explore the interpretability of YoloMal-XAI, we present visualizations of the explanations provided by various XAI heatmap methods across malware categories, along with a quantitative comparison of their performance.

Figure 10 presents a comparative visualization of heatmaps generated by GradCAM, GradCAM++, EigenCAM, and HiResCAM for YOLO11n across five malware categories

(Adware, Banking, Riskware, SMS, Benign) on the CICMalDroid2020 dataset, organized as a grid with rows representing XAI methods and columns corresponding to each category.

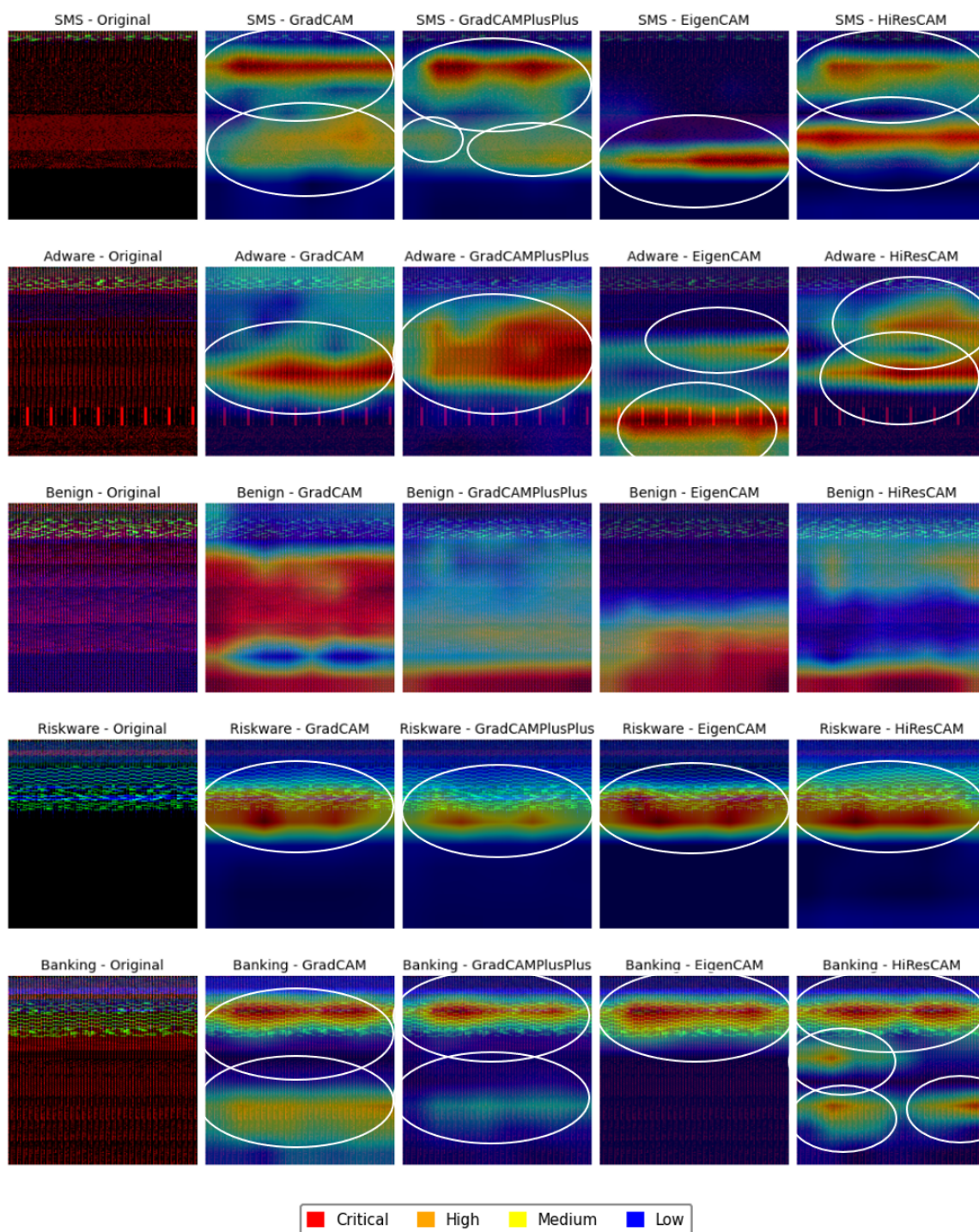


Figure 10. Comparison of heatmap methods across malware categories. Color intensity indicates attention strength: critical (red), high (orange), medium (yellow), and low (blue).

The heatmaps illustrate distinct attention patterns, showcasing how each XAI technique identifies critical regions for classification. GradCAM and GradCAM++ produce broad, diffuse activations with red-to-yellow gradients—especially prominent in SMS and Banking—indicating attention to dominant clusters such as repetitive bytecode sequences. EigenCAM emphasizes more structured vertical patterns, particularly in Adware, which may correspond to sequential permission declarations in the Manifest. HiResCAM of-

fers the most spatially precise maps, with sharper focus regions in Riskware, suggesting localized feature interactions that are crucial for detection.

Benign samples reveal compact attention zones, likely due to their simpler and more uniform structure, while Riskware and SMS exhibit more widespread activations, reflecting their structural complexity. The RGB encoding further aids interpretation: red (DEX) consistently dominates in SMS and Riskware, green (Manifest) stands out in Adware, and blue (Resources) becomes most relevant in Banking. These patterns confirm the DEX segment as the most influential component across categories, with Manifest and Resources playing complementary roles depending on malware type.

To quantify these visual insights, Table 16 evaluates the heatmap methods across six interpretability metrics: faithfulness, flipping, localization accuracy, robustness, sparsity, and runtime efficiency.

Table 16. Quantitative interpretability metrics of heatmap methods in YoloMal-XAI.

Heatmap Method	Faithfulness	Flipping	Localization Accuracy	Robustness	Sparsity	Runtime Efficiency (s)
GradCAM	0.3733	0.2683	0.7800	0.9023	0.9617	0.1301
GradCAM++	0.3069	0.2662	0.7717	0.8878	0.9998	0.1644
EigenCAM	0.2255	0.5452	0.8100	0.8597	0.5219	0.1569
HiResCAM	0.3013	0.2859	0.8447	0.8902	0.9258	0.1039

HiResCAM stands out with the highest localization accuracy (0.8447) and runtime efficiency (0.1039s), aligning with its refined focus on Riskware, while GradCAM++ achieves the highest sparsity (0.9998), supporting its concentrated attention in categories like Banking, though its faithfulness (0.3069) is lower than GradCAM's (0.3733), which offers reliable feature attribution across SMS and other categories. EigenCAM, despite strong localization accuracy (0.8100) in Adware, shows the lowest faithfulness (0.2255) and robustness (0.8597), suggesting potential limitations in diverse scenarios. These quantitative results complement the visual patterns, reinforcing HiResCAM and GradCAM as optimal choices for enhancing YoloMal-XAI's interpretability and supporting its high classification accuracy (99.56% multi-class).

Collectively, these findings confirm that the DEX file is the primary component harboring malware features, as evidenced by the consistent dominance of the red channel across categories like SMS and Riskware, aligning with YOLO11n's effectiveness in detecting code-driven malicious behaviors in Android applications.

5. Discussion

YoloMal-XAI exhibits strong potential for practical Android malware detection by combining accuracy, generalizability, efficiency, and interpretability. Across binary and multi-class tasks, YOLO11 variants consistently outperform baseline CNNs (e.g., ResNet-50, MobileNetV2), and deliver up to $3.5\times$ faster training compared to YOLOv8—underscoring their computational advantage for resource-limited environments. Unlike earlier CNN-based approaches that relied heavily on grayscale representations of DEX bytecode [19,28], YoloMal-XAI's RGB multi-component approach achieves higher accuracy (99.87% binary, 99.56% multi-class) and enhances interpretability, surpassing grayscale-based methods.

Ablation studies highlight the complementary contributions of DEX, Manifest, and Resources components. While DEX alone captures executable behavior effectively [15,17], integrating all three into an RGB image yields the best performance, similar to multi-component fusion approaches [20]. This multi-view representation enhances robustness and reflects real-world complexity across different malware types.

Cross-corpus validation using Drebin and CICAndMal2017 confirms that YOLO11 generalizes well to unseen samples, achieving up to 100% accuracy without retraining. This demonstrates resilience against dataset-specific overfitting and strengthens confidence in deployment scenarios involving diverse app ecosystems. This outperforms models like TuneDroid [17] and DTDroid [18], which rely on single-feature or runtime behavior analysis.

Interpretability analyses using Grad-CAM variants and HiResCAM show that DEX content (red channel) predominantly drives predictions, particularly for SMS and Riskware. Manifest (green) and Resources (blue) contribute more to Adware and Banking malware, respectively. While Grad-CAM has been previously applied to RGB malware images [24], HiResCAM achieved the highest localization IoU (0.8447), confirming its value for forensic analysis and decision transparency.

Although lightweight models such as YOLO11n (1.6 M parameters) show strong potential for mobile deployment, their practical integration into Android environments necessitates further evaluation—particularly regarding inference latency, memory footprint, and resilience to obfuscation or encryption. These aspects are left for future exploration.

Security-wise, adversarial manipulation is a critical challenge. Byte-level perturbations could mislead predictions or disrupt explanations. Although robustness was not explored here, it is a key priority for future work, including adversarial training and defense-aware architectures. Explainability further supports ethical deployment by increasing transparency and minimizing unintended harm from misclassification.

Despite promising results, the primary dataset (CICMalDroid2020) may not fully reflect malware diversity. Although cross-corpus testing mitigates this limitation, broader validation across larger and more heterogeneous datasets is needed.

In future work, we will enhance YoloMal-XAI's adversarial robustness, analyze interpretability under attack conditions, and investigate on-device deployment. We also plan to expand evaluation to large-scale, multi-family malware datasets to better assess scalability and real-world readiness.

6. Conclusions

This work introduced YoloMal-XAI, an explainable deep learning framework for Android malware classification that integrates YOLO11 variants with RGB-based APK representations and interpretable heatmap visualizations. The proposed method achieved outstanding results, with 99.87% accuracy and 99.92% F1-score in binary classification, and 99.56% accuracy and F1-score in multi-class classification—surpassing state-of-the-art CNNs like ResNet-50 and MobileNetV2, as well as recent YOLOv8 benchmarks, while maintaining faster training times and lightweight architectures.

Through extensive ablation studies, we demonstrated the importance of integrating DEX, Manifest, and Resources components. Cross-corpus validation on Drebin and CICAndMal2017 confirmed YoloMal-XAI's ability to generalize to unseen samples, achieving perfect classification in binary settings. Explainability analyses further revealed that the DEX segment drives most decisions, while HiResCAM offered the most accurate and interpretable heatmaps for malware localization.

In future work, we will prioritize optimizing the framework for real-time mobile deployment, strengthening adversarial robustness, and expanding evaluation across larger, multi-family malware corpora. These efforts aim to enhance both the scalability and reliability of YoloMal-XAI in real-world conditions.

Ultimately, YoloMal-XAI offers a practical and interpretable malware detection solution that could be integrated into mobile antivirus software or embedded in Android security layers, contributing to more resilient and transparent malware defense ecosystems.

Author Contributions: Conceptualization, C.E.Y.; methodology, C.E.Y.; software, C.E.Y.; validation, C.E.Y. and K.C.; formal analysis, C.E.Y.; investigation, C.E.Y.; resources, C.E.Y.; data curation, C.E.Y.; writing—original draft preparation, C.E.Y.; writing—review and editing, C.E.Y. and K.C.; visualization, C.E.Y.; supervision, K.C.; project administration, K.C.; funding acquisition, K.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Centre National pour la Recherche Scientifique et Technique (CNRS) under the “PhD-Associate Scholarship-PASS” Program.

Data Availability Statement: The data used in this study are available from the authors upon reasonable request.

Acknowledgments: The authors sincerely thank the Engineering Sciences Laboratory at the National School of Applied Sciences, Ibn Tofail University, Kenitra, Morocco, for providing essential resources and support throughout this research. Additionally, the authors gratefully acknowledge the financial assistance from the CNRS (Centre National pour la Recherche Scientifique et Technique) through the “PhD-Associate Scholarship-PASS” Program, which significantly contributed to the completion of this study.

Conflicts of Interest: The authors have no conflicts of interest to declare.

Abbreviations

The following abbreviations are used in this manuscript:

APK	Android Package Kit
DEX	Dalvik Executable
RGB	Red Green Blue
YOLO	You Only Look Once
XAI	Explainable Artificial Intelligence

References

1. What’s Android’s Market Share? (Updated Jan 2025). Available online: <https://soax.com/research/android-market-share> (accessed on 15 July 2025).
2. M., G.; Sethuraman, S.C. A Comprehensive Survey on Deep Learning Based Malware Detection Techniques. *Comput. Sci. Rev.* **2023**, *47*, 100529. [CrossRef]
3. Si Tienes Esta Aplicación en tu Android la Estafa ha Comenzado. ElHuffPost. Available online: <https://www.huffingtonpost.es/tecnologia/si-tienes-aplicacion-android-estafa-comenzado.html> (accessed on 15 July 2025).
4. New Android Spyware Warning—Do Not Install This App on Your Phone. Available online: <https://www.forbes.com/sites/zakdoffman/2025/01/04/new-android-spyware-warning-do-not-install-this-app-on-your-phone/> (accessed on 15 July 2025).
5. Catalán, C.C. Descubren una Empresa que ha Distribuido Software Espía a Través de Aplicaciones Android Durante Años. Meristation. Available online: <https://as.com/meristation/betech/descubren-una-empresa-que-ha-distribuido-software-espia-a-traves-de-aplicaciones-android-durante-anos-n/> (accessed on 15 July 2025).
6. Ruiz, D. Phishing Evolves Beyond Email to Become Latest Android App Threat. Malwarebytes. Available online: <https://www.malwarebytes.com/blog/news/2025/02/phishing-evolves-beyond-email-to-become-latest-android-app-threat> (accessed on 15 July 2025).
7. Sihag, V.; Vardhan, M.; Singh, P. A Survey of Android Application and Malware Hardening. *Comput. Sci. Rev.* **2021**, *39*, 100365. [CrossRef]
8. Chaurasia, P. Dynamic Analysis of Android Malware Using DroidBox. Master’s Thesis, Tennessee State University, Nashville, TN, USA, 2015; pp. 1–77.
9. Haidros Rahima Manzil, H.; Manohar Naik, S. Detection Approaches for Android Malware: Taxonomy and Review Analysis. *Expert Syst. Appl.* **2024**, *238*, 122255. [CrossRef]
10. Smmarwar, S.K.; Gupta, G.P.; Kumar, S. Android Malware Detection and Identification Frameworks by Leveraging the Machine and Deep Learning Techniques: A Comprehensive Review. *Telemat. Inform. Rep.* **2024**, *14*, 100130. [CrossRef]
11. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware Images: Visualization and Automatic Classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security, Pittsburgh, PA, USA, 20 July 2011; VizSec ’11; Association for Computing Machinery: New York, NY, USA, 2011; pp. 1–7. [CrossRef]

12. Wang, Z.; Liu, Q.; Chi, Y. Review of Android Malware Detection Based on Deep Learning. *IEEE Access* **2020**, *8*, 181102–181126. [CrossRef]
13. Sun, T.; Daoudi, N.; Allix, K.; Samhi, J.; Kim, K.; Zhou, X.; Kabore, A.K.; Kim, D.; Lo, D.; Bissyandé, T.F.; et al. Android Malware Detection Based on Novel Representations of Apps. In *Malware: Handbook of Prevention and Detection*; Gritzalis, D., Choo, K.-K.R., Patsakis, C., Eds.; Springer Nature: Cham, Switzerland, 2025; pp. 197–212. [CrossRef]
14. Casolare, R.; Ciaramella, G.; Iadarola, G.; Martinelli, F.; Mercaldo, F.; Santone, A.; Tommasone, M. On the Resilience of Shallow Machine Learning Classification in Image-Based Malware Detection. *Procedia Comput. Sci.* **2022**, *207*, 145–157. [CrossRef]
15. Daoudi, N.; Samhi, J.; Kabore, A.K.; Allix, K.; Bissyandé, T.F.; Klein, J. DexRay: A Simple, yet Effective Deep Learning Approach to Android Malware Detection Based on Image Representation of Bytecode. In *Deployable Machine Learning for Security Defense*; Wang, G., Ciptadi, A., Ahmadzadeh, A., Eds.; Springer International Publishing: Cham, Switzerland, 2021; pp. 81–106. [CrossRef]
16. Mercaldo, F.; Martinelli, F.; Santone, A. Deep Convolutional Generative Adversarial Networks in Image-Based Android Malware Detection. *Computers* **2024**, *13*, 154. [CrossRef]
17. Bakır, H. A New Method for Tuning the CNN Pre-Trained Models as a Feature Extractor for Malware Detection. *Pattern Anal. Appl.* **2025**, *28*, 26. [CrossRef]
18. Tang, J.; Zhou, S.; Peng, T.; Yan, X.; Hu, X.; Tian, W. DTDroid: Adversarial Packed Android Malware Detection Based on Traffic and Dynamic Behavioral. *IEEE Internet Things J.* **2025**, *12*, 2646–2658. [CrossRef]
19. Chaymae, E.Y.; Khalid, C. Android Malware Detection Through CNN Ensemble Learning on Grayscale Images. *Int. J. Adv. Comput. Sci. Appl. (IJACSA)* **2025**, *16*, 11. [CrossRef]
20. Chaymae, E.Y.; Khalid, C. Image-Based Approach for Android Malware Detection Using APK Component Fusion and Deep Learning. In Proceedings of the 2025 5th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET), Fez, Morocco, 15–16 May 2025; pp. 1–7. [CrossRef]
21. Vasan, D.; Alazab, M.; Wassan, S.; Naeem, H.; Safaei, B.; Zheng, Q. IMCFN: Image-Based Malware Classification Using Fine-Tuned Convolutional Neural Network Architecture. *Comput. Netw.* **2020**, *171*, 107138. [CrossRef]
22. Yadav, P.; Menon, N.; Ravi, V.; Vishvanathan, S.; Pham, T.D. EfficientNet Convolutional Neural Networks-Based Android Malware Detection. *Comput. Secur.* **2022**, *115*, 102622. [CrossRef]
23. Yadav, P.; Menon, N.; Ravi, V.; Vishvanathan, S.; Pham, T.D. A Two-Stage Deep Learning Framework for Image-Based Android Malware Detection and Variant Classification. *Comput. Intell.* **2022**, *38*, 1748–1771. [CrossRef]
24. Naeem, H.; Alshammari, B.M.; Ullah, F. Explainable Artificial Intelligence-Based IoT Device Malware Detection Mechanism Using Image Visualization and Fine-Tuned CNN-Based Transfer Learning Model. *Comput. Intell. Neurosci.* **2022**, *2022*, 7671967. [CrossRef]
25. Tasyurek, M.; Arslan, R.S. RT-Droid: A Novel Approach for Real-Time Android Application Analysis with Transfer Learning-Based CNN Models. *J. Real-Time Image Process.* **2023**, *20*, 55. [CrossRef]
26. Ksibi, A.; Zakariah, M.; Almuqren, L.; Alluhaidan, A. Deep Convolution Neural Networks for Image-Based Android Malware Classification. *Comput. Mater. Contin.* **2025**, *82*, 4093–4116. [CrossRef]
27. Li, X.; Liu, L.; Liu, Y.; Liu, H. Detecting Android Malware: A Multimodal Fusion Method with Fine-Grained Feature. *Inf. Fusion* **2025**, *114*, 102662. [CrossRef]
28. Wasif, M.S.; Miah, M.P.; Hossain, M.S.; Alenazi, M.J.F.; Atiquzzaman, M. CNN-ViT Synergy: An Efficient Android Malware Detection Approach through Deep Learning. *Comput. Electr. Eng.* **2025**, *123*, 110039. [CrossRef]
29. Ehsan, A.; Catal, C.; Mishra, A. Detecting Malware by Analyzing App Permissions on Android Platform: A Systematic Literature Review. *Sensors* **2022**, *22*, 7928. [CrossRef]
30. Yerima, S.Y.; Sezer, S.; Muttik, I. Android Malware Detection Using Parallel Machine Learning Classifiers. In Proceedings of the 2014 Eighth International Conference on Next Generation Mobile Apps, Services and Technologies, Oxford, UK, 10–12 September 2014; pp. 37–42. [CrossRef]
31. Afonso, V.; Bianchi, A.; Fratantonio, Y.; Doupe, A.; Polino, M.; De Geus, P.; Kruegel, C.; Vigna, G. Going Native: Using a Large-Scale Analysis of Android Apps to Create a Practical Native-Code Sandboxing Policy. In Proceedings of the 2016 Network and Distributed System Security Symposium, San Diego, CA, USA, 21–24 February 2016; Internet Society: San Diego, CA, USA, 2016. [CrossRef]
32. Ultralytics. Computer Vision Tasks Supported by Ultralytics YOLOv11. Available online: <https://docs.ultralytics.com/tasks> (accessed on 15 July 2025).
33. Khanam, R.; Hussain, M. YOLOv11: An Overview of the Key Architectural Enhancements. *arXiv* **2024**. [CrossRef]
34. Alsubai, S.; Dutta, A.K.; Alnajim, A.M.; Sait, A.; Rahaman, W.; Ayub, R.; AlShehri, A.M.; Ahmad, N. Artificial Intelligence-Driven Malware Detection Framework for Internet of Things Environment. *PeerJ Comput. Sci.* **2023**, *9*, e1366. [CrossRef] [PubMed]
35. Ultralytics. Classify. Available online: <https://docs.ultralytics.com/tasks/classify> (accessed on 15 July 2025).
36. Gildenblat, J. Jacobgil/Pytorch-Grad-Cam. 2025. Available online: <https://github.com/jacobgil/pytorch-grad-cam> (accessed on 14 July 2025).

37. Baniecki, H.; Biecek, P. Adversarial Attacks and Defenses in Explainable Artificial Intelligence: A Survey. *Inf. Fusion* **2024**, *107*, 102303. [[CrossRef](#)]
38. MahdaviFar, S.; Abdul Kadir, A.F.; Fatemi, R.; Alhadidi, D.; Ghorbani, A.A. Dynamic Android Malware Category Classification Using Semi-Supervised Deep Learning. In Proceedings of the 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech), Calgary, AB, Canada, 17–22 August 2020; pp. 515–522. [[CrossRef](#)]
39. Arp, D.; Spreitzenbarth, M.; Hübner, M.; Gascon, H.; Rieck, K. Drebin: Effective and Explainable Detection of Android Malware in Your Pocket. In Proceedings of the 2014 Network and Distributed System Security Symposium, San Diego, CA, USA, 23–26 February 2014; Internet Society: San Diego, CA, USA, 2014. [[CrossRef](#)]
40. Lashkari, A.H.; Kadir, A.F.A.; Taheri, L.; Ghorbani, A.A. Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification. In Proceedings of the 2018 International Carnahan Conference on Security Technology (ICCST), Montreal, QC, Canada, 22–25 October 2018; pp. 1–7. [[CrossRef](#)]
41. Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. In Proceedings of the 7th International Conference on Learning Representations (ICLR 2019), New Orleans, LA, USA, 6–9 May 2019; OpenReview.net: Alameda, CA, USA, 2019.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.